

STARVNT

Complete Product & Engineering Structure

One connected Event Operating Platform — from customer intent to planning, vendor fulfilment, payment, execution, settlement and event memory.

REUSE → INTEGRATE → TEST → PROVE

Executive Product Direction

STARVNT is being built as one connected platform. The areas in this document are functional/product domains, not separate products, independent platforms, or parallel backend systems.

The purpose is to eliminate ambiguity, prevent overlapping development, avoid duplicate systems, and align every development decision with one overall architecture.

Customer principle

> **“The customer should tell STARVNT what they want. STARVNT should figure out what is required to make it happen.”**

Customer Journey
Imagine → Tell → Understand → Plan → Discover → Compare → Decide → Book → Pay → Coordinate → Track → Complete

1. Customer Experience

Customer journey and interaction layer.

- Customer Website
- Customer App
- Natural-language event planning
- Voice and text interaction
- Event creation
- Event Plan
- Requirement discovery and clarification
- Vendor discovery
- Vendor comparison
- Quote review
- Booking
- Payment experience
- Event tracking
- Event Circle
- Event Day
- Live Event
- Event Memory

2. Event & Core Platform

Central business foundation and official source of truth.

- Customer
- Organization
- Event
- Event Type
- Event Destination
- Venue

- Service Location
- Requirement
- Booking
- Payment
- Execution
- Completion
- Settlement
- Identity
- Authentication
- Permissions
- Core APIs
- Business State Management
- Security
- Tenant/Data Isolation

The Core remains the source of truth for official business states. No other layer should independently create a parallel version of these truths.

3. Vendor Ecosystem / Universal Vendor OS

One universal vendor ecosystem for all vendor categories.

- Vendor Profile
- Category
- Services
- Capability
- Capacity
- Service Location
- Coverage
- Availability
- Pricing
- Packages
- Commercial Terms
- Portfolio
- Enquiries
- Quotes
- Quote Responses
- Booking Information
- Execution Information
- Completion Evidence
- Vendor Business Management

Vendor discovery: Requirement → Service Location → Coverage → Availability → Capacity → Eligibility → Cost → Suitability → Vendor Options

The objective is the **lowest validated total-cost option** that satisfies the requirement and can operationally deliver it.

4. Revenue & Customer Intelligence

Demand-side business and revenue workflow.

- Revenue CRM
- Lead Management
- Customer Management
- Sales Activity
- Follow-up
- Pipeline
- Revenue Intelligence
- Conversion Intelligence
- Customer Intelligence

5. AI / Aura+ Intelligence

Intelligence layer: Understand → Analyse → Recommend → Explain → Assist.

- Natural Language Understanding
- Voice/Text Understanding
- Intent Detection
- Requirement Understanding
- Missing Requirement Detection
- Event Planning Assistance
- Vendor Comparison
- Recommendation
- Budget Intelligence
- Risk Detection
- Next-action Guidance

Aura+ is an intelligence layer, not the business authority. It does not become the source of truth for vendor eligibility, booking, payment, settlement or deterministic commercial calculations.

6. Automation & Workflow

Execution layer for authorized automated workflows.

- Lead follow-up
- Vendor enquiry
- Quote notification
- Booking workflow
- Payment workflow
- Execution reminders

- Completion workflow
- Settlement workflow
- Notifications
- Scheduled actions
- Retry handling
- Idempotency
- Failure recovery
- Audit
- Activity tracking

7. Communication & Event Circle

Controlled communication and event-social layer around real events.

- Secure Communication
- Event Chat
- Event Updates
- Event Circle
- Customer/Vendor Coordination
- Photos
- Videos
- Event Milestones
- Event Day Communication
- Live Event
- Event Memory
- Privacy & Access Control

8. Commercial & Payment

Transaction lifecycle: Quote → Approval → Booking → Payment → Verification → Execution → Completion → Settlement.

- Pricing
- Quote
- Minimum Booking Amount
- Payment Gateway
- Server-side Payment Verification
- Payment Transaction
- Refund / Recovery
- Completion Validation
- Settlement Eligibility
- Vendor Settlement
- Commercial Records

Customer Payment ≠ Vendor Settlement

Payment confirmation, execution, completion verification, settlement eligibility and settlement remain distinct lifecycle states.

9. Data, Security & Reliability

Common technical trust layer for the platform.

- Authentication
- Authorization
- RBAC
- Tenant Isolation
- Data Protection
- API Security
- Audit
- Logging
- Monitoring
- Error Handling
- Idempotency
- Retry
- Outbox
- Worker Reliability
- Failure Recovery

10. Overall STARVNT Architecture

All functional areas connect through one common architecture and one commercial/execution lifecycle.

CUSTOMER SIDE	BUSINESS CORE	INTELLIGENCE
Customer Experience	Event & Core	Aura+ Intelligence
Event Circle	Vendor Ecosystem	Revenue Intelligence
COMMERCIAL	**EXECUTION**	**TRUST**
Quote → Booking → Payment	Automation → Completion → Settlement	Data / Security / Audit

One connected platform:

- One Customer Experience
- One Core Platform
- One Vendor Ecosystem
- One Revenue System
- One Aura+ Intelligence Layer
- One Automation Layer
- One Commercial/Payment Flow
- One Event Circle

No duplicate Core, CRM, Automation Engine, Booking System, Payment System or parallel business-truth database.

11. AI Infrastructure Decision

MVP: No self-hosted AI model and no continuously running dedicated AWS GPU infrastructure.

The MVP will use third-party AI APIs such as OpenAI API / Gemini API using a usage-based / pay-per-use model.

Future self-hosting can be evaluated separately if scale and economics justify it. It is not an MVP development dependency.

API usage may be variable, while normal backend, database, storage, hosting and monitoring costs still exist. The decision is specifically that dedicated continuously running GPU model hosting is not required for the MVP.

12. AI vs Core — Clear Boundary

Core + Database + Deterministic Logic

- Search
- Vendor Discovery
- Location Matching
- Service Location Matching
- Availability
- Capacity

- Eligibility
- Vendor Filtering
- Total Cost Calculation
- Booking State
- Payment State
- Execution State
- Completion State
- Settlement State

Aura+

- Understanding user intent
- Natural language understanding
- Missing information identification
- Requirement structuring assistance
- Explanation
- Comparison
- Recommendations
- Budget/risk insights
- Next-action guidance

Aura+ helps understand, analyse and recommend. Core decides and records what is officially true.

13. Revenue CRM + Vendor Ecosystem

Demand Side: Lead → Qualification → Follow-up → Sales → Conversion → Revenue

Supply / Fulfilment Side: Requirement → Vendor → Enquiry → Quote → Booking → Execution

These systems ultimately connect through the Core platform. No duplicate CRM. No parallel Vendor system. No parallel Core.

14. Development Principles

REUSE — First inspect what already exists.

INTEGRATE — Use existing platform and contracts wherever applicable.

EXTEND — Extend existing capability rather than creating a parallel implementation.

TEST — Test real application states and realistic E2E flows.

PROVE — Show working evidence through application/API/database/workflow, not only documentation or screenshots.

> **The objective is not to maximise code. The objective is to build the smallest connected system that genuinely works.**

15. Development Planning — Required Inputs

The AI infrastructure decision is already clear. Development estimates should now be based on actual product scope, integration work, team capacity and delivery requirements.

A. Vendor OS

- Current status
- Completed scope
- Remaining scope
- Remaining development effort
- Expected completion date

B. Core Integration

- Existing Core components requiring integration
- Required APIs / events / contracts
- Integration effort
- Testing effort
- Full E2E proof timeline

C. Team Structure

- Required development capacity
- Proposed team structure
- Each person's planned scope
- Proposed freelancer role
- Why the freelancer is required
- Expected duration

D. Budget

- Monthly development cost
- Proposed freelancer cost
- Expected monthly team budget
- Expected MVP development budget
- Any proposed fixed overall MVP budget

E. Delivery

- First usable milestone
- Next major milestone
- MVP target date
- Major dependencies
- Actual blockers

I need realistic dates and numbers, not an open-ended estimate.

16. Freelancer Approval

If an additional freelancer is proposed, provide **Name** → **Role** → **Scope** → **Duration** → **Weekly/Monthly Cost** before any financial commitment.

No new freelancer-related financial commitment should be made without explicit approval. Existing approved development work can continue; any new resource commitment should be separately approved.

17. Final Product Principle

STARVNT is not merely a Vendor Dashboard, Customer Dashboard, CRM or AI Chatbot. It is a connected Event Operating Platform.

The customer should be able to describe what they want naturally, and STARVNT should progressively convert that intent into a structured, actionable and trackable event.

Customer

Understand

Plan

Requirements

Vendor Ecosystem

Matching

Quote

Approval

Booking

Payment

Execution

Completion

Settlement

Event Memory

The development objective is not to build isolated modules. It is to build one connected system where every functional area works together through clearly defined boundaries and shared business truth.

NON-NEGOTIABLES

- No duplicate system.
- No unnecessary infrastructure.
- No open-ended scope.
- No parallel business truth.
- No development without integration.

REUSE → INTEGRATE → TEST → PROVE

Next step: lock the timeline + team structure + monthly/MVP budget + delivery milestones based on this product scope.